

PRISM-DSA: A Concrete Fixed-Iteration Signing Construction for ML-DSA — Implementation, Distributional Equivalence Proof, and Benchmark

Draft — Research Preprint

Not peer reviewed. Submitted to IACR ePrint as a research draft.

Abstract

We present PRISM-DSA, a post-quantum digital signature scheme that eliminates the **rejection-count timing channel** from Fiat-Shamir with Aborts (FSwA) signatures. Standard ML-DSA (FIPS 204) terminates signing via a geometrically distributed rejection loop; an adversary observing total signing time can estimate the iteration count, and this variable-time behavior constitutes a leakage source that, when combined with power/EM/cache measurements, can amplify key recovery attacks [Ravi+24, Berzati+23, Ulitzsch+24]. PRISM-DSA replaces this loop with a Fixed-Iteration Signing (FIS) construction that always performs exactly $\text{FIS_SLOTS} = 64$ attempts, using constant-time conditional selection (via the `subtle` crate’s CMOV abstraction) to output the first valid result without secret-dependent control flow.

Scope of the “constant-time” claim. FIS eliminates the rejection-count timing channel specifically. Two residual channels remain (NTT butterfly arithmetic and SHAKE-256 input scheduling) and are documented in Section 5.4. Additionally, FIS does not affect the distribution of the public hint polynomial weight $\text{wt}(h)$: empirical measurements confirm that $E[\text{wt}(h)]$ is key-dependent under PRISM-DSA ($F=8.94$, $p \approx 10^{-63}$, 50 keys $\times$ 2,000 sigs) at the same level as ML-DSA-44 — this is a structural property of `MakeHint` and is independent of the signing loop design. See Section 4.5 and [COBALT26].

The scheme uses identical parameters to ML-DSA-44/65/87; PRISM-DSA signatures are valid ML-DSA signatures. On an aarch64 Linux machine (pure Rust, no SIMD, Rust-vs-Rust comparison against RustCrypto ml-dsa), PRISM-128 achieves keygen in 69.4 μ s (comparable to ML-DSA-44 at 67.6 μ s, faster than ML-DSA-65 at 109.7 μ s), signing in 1.63 ms, and verification in 49.4 μ s. The signing overhead vs non-CT ML-DSA-44 (same $K=4$, $L=4$ dimensions) is $10\times$; however, a constant-time ML-DSA-44 forced to run all 64 iterations would cost ≈ 2.3 ms, making PRISM-128 **$1.4\times$ faster** for the same CT guarantee. The failure probability is 2^{-23} at $\text{FIS_SLOTS} = 64$ (approximately 1 failure per 8 million signing operations).

1. Introduction

1.1 Motivation

The transition to post-quantum cryptography, accelerated by NIST's selection of ML-DSA [FIPS204] as the primary post-quantum digital signature standard, introduces new implementation security concerns. While ML-DSA's security against quantum computers is well-analyzed, its signing algorithm contains an inherent source of microarchitectural leakage: the **rejection sampling loop**.

Fiat-Shamir with Aborts (FSwA) [Lyub09] is the fundamental paradigm underlying lattice-based signatures including Dilithium [BDLOP18] and its standardized descendant ML-DSA. The protocol signs a message by:

1. Sampling a random mask vector y
2. Computing a commitment $w = Ay$
3. Generating a challenge c from the commitment
4. Computing the response $z = y + c \cdot s_1$
5. **Rejecting and repeating** if z leaks too much information about s_1

The rejection step is essential for security: without it, $z = y + c \cdot s_1$ would be a Gaussian-sampled value from which s_1 could be extracted. The rejection ensures z is drawn from a distribution that is statistically independent of s_1 .

However, this rejection loop is **variable-time**. The number of iterations before success is a geometrically distributed random variable with success probability $p \approx 0.22$ per attempt (for PRISM-128 / ML-DSA-44 parameters). An attacker with precise timing measurements can:

1. Estimate the number of iterations from the total signing time

2. Correlate iteration counts with key-dependent rejection probabilities
3. Mount a statistical attack to recover portions of s_1 or s_2 [Ravi+24, Ulitzsch+24]

Side-channel attacks on lattice schemes that exploit variable-time signing behavior have been demonstrated using power/EM/cache analysis [Ravi+24, Berzati+23, Ulitzsch+24]. Note: these works require hardware instrumentation (power traces, profiling attacks); they are not pure wall-clock timing attacks. However, variable-loop termination provides a timing observable that can serve as an additional signal in such attacks, particularly on shared infrastructure (cloud VMs, co-located processes, speculative execution environments).

Beyond timing, variable-loop termination also complicates formal verification: standard program verification tools handle unbounded loops poorly, and existing formal proofs of ML-DSA [KLS18] require careful treatment of the loop invariant and termination argument.

1.2 Our Contribution

We present **PRISM-DSA** (Post-quantum Ring-based Ideal Signature Mechanism with Fixed-Iteration Signing), a concrete, benchmarked instantiation of fixed-iteration signing for all three ML-DSA parameter sets.

Fixed-Iteration Signing (FIS): The pattern of running a fixed number of signing attempts and using a constant-time conditional move to select the first valid result is a known technique in constant-time cryptography (related to Falcon’s constant-time Gaussian sampler [FALCON §3.12] and noted as a Dilithium mitigation in [Azouaoui+23]). Our contribution is not the pattern itself, but: (a) a **concrete, benchmarked Rust implementation** for all three ML-DSA variants, (b) an **explicit distributional equivalence proof** (Theorem 4.1) with a formal statistical distance bound, (c) a **systematic FIS_SLOTS parameter analysis** documenting the failure probability / performance tradeoff, and (d) an empirical cross-reference showing FIS does not eliminate the hint weight leakage channel identified in [COBALT26].

The FIS construction: Instead of looping until a valid signature is found, FIS executes exactly $\text{FIS_SLOTS} = 64$ signing attempts. All attempts are computed fully and unconditionally. The first valid slot is selected using a constant-time conditional move instruction (via Rust’s `subtle` crate), without any secret-dependent branching.

Key properties: - Signing time is exactly $\text{FIS_SLOTS} \times$ (one attempt), regardless of key, message, or randomness, modulo the NTT and SHAKE-256 residual channels documented in §5.4 —

rejection-count timing channel eliminated - All norm checks are constant-time (no early exit) - The output distribution is statistically close to ML-DSA with distance $\leq 2^{-23}$ (conditional on success; the output distribution of the first valid slot is independent of the index of that slot — see Theorem 4.1) - Failure probability is at most 2^{-23} for PRISM-128 (FIS_SLOTS = 64) ≈ 1 in 8×10^6 - **Does not affect** the hint polynomial weight distribution (structural ML-DSA property)

Compatibility: PRISM-DSA is parameter-compatible with ML-DSA-44 (PRISM-128), ML-DSA-65 (PRISM-192), and ML-DSA-87 (PRISM-256). PRISM-DSA signatures are valid ML-DSA signatures and pass ML-DSA verification unchanged. Key generation and verification are identical to ML-DSA.

Implementation: A pure-Rust implementation is provided with no unsafe code, using the `subtle` crate for constant-time selection and `rand_core::CryptoRng` for randomness. Benchmarks are provided for all three variants.

1.3 Paper Organization

Section 2 covers background on ML-DSA, FSwA, and rejection sampling. Section 3 gives a formal description of the FIS construction. Section 4 analyzes security. Section 5 describes our Rust implementation and its constant-time audit. Section 6 discusses parameter selection for FIS_SLOTS. Section 7 compares to related work. Section 8 concludes with open problems.

2. Background

2.1 Module Lattices and Hard Problems

All schemes in this work operate over the ring $R_q = \mathbb{Z}_q[X]/(X^{N+1})$ with $N = 256$ and $q = 8380417$. Module lattices are structured lattices defined as images of modules over R_q , combining the algebraic structure of ring lattices with the flexibility of unstructured lattices.

Definition 2.1 (Module-SIS_{q,K,L,β}): Given $A \leftarrow R_q^{K \times L}$ uniformly at random, find a nonzero vector $z \in R_q^{K+L}$ with $\|z\|_\infty \leq \beta$ and $[A|I_K] \cdot z = 0 \pmod{q}$.

Definition 2.2 (Module-LWE_{q,K,L,η}): Given $A \leftarrow R_q^{K \times L}$ uniformly at random and $s \leftarrow S_\eta^L$, $e \leftarrow S_\eta^K$, distinguish $(A, As+e)$ from (A, u) where $u \leftarrow R_q^K$ uniform.

Both Module-SIS and Module-LWE are believed hard for polynomial-time quantum algorithms when the parameters satisfy the conditions in [FIPS204, Appendix C] [Pei16]. The best known attacks require solving the Shortest Vector Problem (SVP) in a lattice of dimension proportional to $K \cdot N$ or $L \cdot N$, which is believed to require exponential quantum time [Pei16, FIPS204].

2.2 ML-DSA (FIPS 204) Overview

ML-DSA is the standardized version of CRYSTALS-Dilithium [BDLOP18]. It is a Fiat-Shamir signature scheme with the following structure.

KeyGen: Sample $A \leftarrow R_q^{\{K \times L\}}$ from seed ρ . Sample short secrets $s_1 \leftarrow S_\eta^L$, $s_2 \leftarrow S_\eta^K$. Compute $t = As_1 + s_2$. Use Power2Round to write $t = t_1 \cdot 2^D + t_0$ and publish $pk = (\rho, t_1)$.

Sign: To sign message m : 1. Compute $\mu = H(H(pk) \parallel m)$ 2. Derive per-signature randomness $\rho' = H(\text{key} \parallel \text{rnd} \parallel \mu)$ 3. Loop: a. Sample $y \leftarrow R_q^L$ with $\|y\|_\infty < \gamma_1$ b. Compute $w = Ay$; decompose into $(w_1, w_0) = \text{Decompose}(w, 2\gamma_2)$ c. Compute $\tilde{c} = H(\mu \parallel w_1)$; $c = \text{SampleInBall}(\tilde{c}, \tau)$ d. Compute $z = y + cs_1$ e. **Reject** (restart loop) if: $\|z\|_\infty \geq \gamma_1 - \beta$, or $\|w_0 - cs_2\|_\infty \geq \gamma_2 - \beta$, or $\|ct_0\|_\infty \geq \gamma_2$, or $\|h\|_1 > \omega$ f. Output $\sigma = (\tilde{c}, z, \text{MakeHint}(w_0 - cs_2 + ct_0, w_1, \gamma_2))$

Verify: Check $\|z\|_\infty < \gamma_1 - \beta$, hint weight $\leq \omega$, and $\tilde{c} = H(\mu \parallel \text{UseHint}(h, Az - ct_1 2^D))$.

2.3 Fiat-Shamir with Aborts and Rejection Sampling

The security of FSswA [Lyub09] rests on rejection sampling to ensure z is independent of the secret s_1 . The acceptance condition $\|z\|_\infty < \gamma_1 - \beta$ ensures that z , viewed as a uniform sample from a cube of radius $\gamma_1 - \beta$, is statistically close to the conditional distribution $z \mid (z = y + cs_1)$ [Lyub09].

The acceptance probability per iteration is approximately

$$p_{\text{accept}} = \Pr[\|y + cs_1\|_\infty < \gamma_1 - \beta] \times \Pr[\|w_0 - cs_2\|_\infty < \gamma_2 - \beta] \times \dots$$

For ML-DSA-44 / PRISM-128 parameters:

- $\Pr[\|z\|_\infty < \gamma_1 - \beta]$: With $\gamma_1 = 2^{17}$, $\beta = 78$, and y uniform in $(-2^{17}, 2^{17}] \{LN\}$, roughly $78/131072 \approx 0.06\%$ of coordinates are in the “danger zone” per rejection event; the combined probability over all coordinates gives acceptance $\approx 60\%$ [BDLOP18].
- Additional checks reduce this to $p_{\text{accept}} \approx 0.22$ overall.

- Expected iterations: $1/0.22 \approx 4.55$ for PRISM-128.

The key insight is that y is independently sampled each iteration; the rejection does not adaptively modify y based on s_1 . This independence is the basis of the security proof. However, it also means that each iteration is an independent Bernoulli trial, making the loop duration geometrically distributed with rate p_{accept} .

2.4 Timing Side-Channels in FSwa

The total signing time for ML-DSA is roughly:

$$T_{\text{sign}} \approx \text{Geometric}(p_{\text{accept}}) \times T_{\text{one_iteration}}$$

where $T_{\text{one_iteration}}$ is approximately constant. An adversary observing T_{sign} can estimate the number of iterations $k = T_{\text{sign}} / T_{\text{one_iteration}}$ and use this to infer statistical information about the secret key [Ravi+24].

More precisely, the acceptance probability p_{accept} is a function of the secret key parameters η , τ , β and the randomness y . While y is not directly secret, correlations between rejection counts and key-specific properties can be exploited in adaptive chosen-message attacks where the adversary controls message scheduling to maximize information leakage [Ravi+24, Ulitzsch+24].

This concern is not merely theoretical. Timing attacks on Dilithium (the predecessor to ML-DSA) have been documented [Ravi+24, Berzati+23, Ulitzsch+24], motivating the need for constant-time signing.

3. The FIS Construction

3.1 Fixed-Iteration Signing Algorithm

The FIS construction is parameterized by $\text{FIS_SLOTS} \in \mathbb{N}^+$. Let T be the set of inputs $(sk, m, \text{ctx}, \text{rnd})$ and let $\text{Attempt}(sk, m, \rho', i)$ denote the i -th signing attempt derived from per-signature randomness ρ' .

Definition 3.1 (FIS-Sign): On input (sk, m, ctx) with $\text{rnd} \leftarrow \{0,1\}^{256}$:

```
FIS-Sign( $sk, m, \text{ctx}$ ):
  Derive  $\rho' = H(\text{key} \parallel \text{rnd} \parallel \mu)$ 
  result  $\leftarrow \perp$ ; found  $\leftarrow 0$ 
  for  $i = 0$  to  $\text{FIS\_SLOTS} - 1$ :
```

```

    (valid_i,  $\sigma_i$ )  $\leftarrow$  Attempt(sk, m,  $\rho'$ , i)
    use_this  $\leftarrow$  CT_AND(valid_i, CT_NOT(found))
    result  $\leftarrow$  CT_SELECT(use_this,  $\sigma_i$ , result)
    found  $\leftarrow$  CT_OR(found, valid_i)
    if found = 0: return FAILURE
    return result

```

where CT_AND, CT_NOT, CT_OR, CT_SELECT are constant-time operations that do not branch on the boolean arguments.

Definition 3.2 (Attempt): The i -th attempt computes:

```

Attempt(sk, m,  $\rho'$ , i):
    nonce  $\leftarrow$  i  $\times$  L
    y  $\leftarrow$  ExpandMask( $\rho'$ , nonce,  $\gamma_1$ )
    w  $\leftarrow$  Ay; ( $w_1$ ,  $w_0$ )  $\leftarrow$  Decompose(w,  $2\gamma_2$ )
     $\check{c} \leftarrow H(\mu \parallel w_1)$ ; c  $\leftarrow$  SampleInBall( $\check{c}$ ,  $\tau$ )
    z  $\leftarrow$  y +  $cs_1$ 
    valid  $\leftarrow$  CT_NormCheck(z,  $\gamma_1 - \beta$ )
        & CT_NormCheck( $w_0 - cs_2$ ,  $\gamma_2 - \beta$ )
        & CT_NormCheck( $ct_0$ ,  $\gamma_2$ )
        & CT_WeightCheck(h,  $\omega$ )
    h  $\leftarrow$  MakeHint( $w_0 - cs_2 + ct_0$ ,  $w_1$ ,  $\gamma_2$ )
    return (valid, Encode( $\check{c}$ , z, h))

```

Note that all computations (cs_1 , cs_2 , ct_0 , h) are performed unconditionally, even when valid = 0. This is essential for constant-time behavior.

3.2 The Constant-Time Selection Argument

The key claim is that CT_SELECT and the associated logic introduce no secret-dependent timing variation. We establish this via three sub-claims:

Claim 3.1 (CT_NormCheck is CT): The function `check_norm(v, bound)` iterates all $N = 256$ coefficients, applying `Reduce32` and checking $|r| < \text{bound}$ without early exit. The `&=` operator accumulates the result into a `bool` (which the Rust compiler guarantees to be false-propagating under optimization). No branch is taken on any intermediate result.

Formal justification: In LLVM IR, `&= (|r| < bound)` compiles to a comparison instruction (ICMP) followed by AND, both of which are single-cycle on modern microarchitectures with no branch predictor involvement.

Claim 3.2 (CT_SELECT is CT): The operation `u8::conditional_select(a, b, choice)` from the `subtle` crate compiles to a CMOV (conditional move) instruction on x86-64, ARM64, and RISC-V. CMOV is a single-cycle non-branching instruction [Intel23, ARM23].

Implementation: The `subtle` crate uses volatile inline assembly or compiler intrinsics to prevent the optimizer from reintroducing a branch from the bitwise select $(a \ \& \ !\text{mask}) \mid (b \ \& \ \text{mask})$.

Claim 3.3 (Loop count is fixed): The for loop for `i` in `0..FIS_SLOTS` iterates exactly `FIS_SLOTS` times unconditionally. The `found` flag never causes a `break` or `continue`. Its value affects only the output via `CT_SELECT` in the subsequent iteration, not the loop control.

Together, Claims 3.1–3.3 imply that the control flow graph of FIS-Sign is independent of the secret key, randomness, and message. The execution time is bounded by a deterministic function of `FIS_SLOTS` and the parameter set.

3.3 Nonce Derivation and Slot Independence

Each slot uses a distinct nonce:

$$\text{nonce_i} = i \times L$$

where `L` is the number of columns in the matrix `A`. The mask vector for slot `i` is:

$$y_{\{ij\}} = \text{ExpandMask}(\rho', i \times L + j, \gamma_1) \quad \text{for } j \in [0, L)$$

Since `ExpandMask` uses SHAKE-256 with distinct inputs $(\rho', (i \times L + j)_{\text{le16}})$ for each polynomial, and SHAKE-256 is modeled as a random oracle, the vectors $(y_0, y_1, \dots, y_{\{\text{FIS_SLOTS}-1\}})$ are computationally indistinguishable from independently uniform samples from $(-\gamma_1, \gamma_1)^{\{L \times N\}}$.

Formally, the nonce spacing of `L` ensures that no two polynomials across different slots share the same SHAKE-256 input, so all $N \times L \times \text{FIS_SLOTS}$ derived polynomials are independently pseudo-random in the random oracle model.

4. Security Analysis

4.1 Output Distribution

The central security claim is that FIS produces signatures with the same distribution as standard ML-DSA rejection sampling, conditional on success.

Theorem 4.1 (Output Distribution): Let $D_{\{\text{ML-DSA}\}}(\text{sk}, m)$ denote the distribution of valid ML-DSA signatures for (sk, m) . Let $D_{\{\text{FIS}\}}(\text{sk}, m)$ denote the distribution of PRISM-DSA FIS signatures conditioned on success ($\text{found} = 1$). Then:

$$\Delta(D_{\{\text{FIS}\}}(\text{sk}, m), D_{\{\text{ML-DSA}\}}(\text{sk}, m)) \leq (1 - p)^{\{\text{FIS_SLOTS}\}}$$

where Δ denotes statistical distance and $p = \Pr[\text{one attempt is valid}]$. For PRISM-128 at $\text{FIS_SLOTS} = 64$ and $p \approx 0.22$: $\Delta \leq 2^{-23}$.

Proof sketch: Slots are independent (Claim 3.3 plus random oracle model for SHAKE-256). Each valid slot produces a signature drawn from $D_{\{\text{ML-DSA}\}}$: the acceptance conditions are identical; the distribution of y — and hence z — given acceptance is the same as in ML-DSA by the rejection sampling argument of [Lyub09].

FIS selects the first valid slot from at most FIS_SLOTS independent attempts. Let $X_1, \dots, X_{\{64\}}$ be i.i.d. Bernoulli(p) validity indicators, $\sigma_1, \dots, \sigma_{\{64\}}$ the corresponding candidates. The first valid candidate σ_T where $T = \min\{i : X_i = 1\}$ satisfies:

$$\begin{aligned} \Pr[\sigma_T \in S] &= \Pr[\sigma_i \in S \mid X_i = 1] = \Pr[\sigma_i \in S \mid \sigma_i \text{ is ML-DSA-valid}] \\ &= D_{\{\text{ML-DSA}\}}(S) \end{aligned}$$

Conditioning caveat: ML-DSA conditions on “loop terminates” (probability 1 by construction). PRISM-DSA conditions on “at least one of 64 slots is valid” (probability $1 - (1-p)^{\{64\}}$). These events differ: PRISM-DSA’s success condition slightly oversamples $(\text{key}, \text{message})$ pairs for which more slots pass, introducing a statistical distance of at most $(1-p)^{\{64\}} = 2^{-23}$. In the regime $(1-p)^{\{64\}} \rightarrow 0$ this bias is negligible. The CT_SELECT operation outputs σ_T without disclosing T via timing. $\square$

Remark 4.1: The proof assumes the random oracle model for SHAKE-256 (specifically, the independence of SHAKE-256 outputs on distinct inputs) and the uniformity of ExpandMask outputs in the ROM. In the standard model, this reduces to a pseudorandomness assumption on SHAKE-256.

4.2 EUF-CMA Security

Theorem 4.2 (Unforgeability): PRISM-DSA is EUF-CMA secure under Module-SIS and Module-LWE, assuming $\text{FIS_SLOTS} \geq 1$.

Proof sketch: We reduce to ML-DSA security. Suppose adversary A breaks PRISM-DSA EUF-CMA: A receives a public key pk and a signing oracle $O_{\{\text{FIS}\}}$, and outputs a forgery (m, σ) with $\sigma^* \notin \{\text{signatures queried for } m^*\}$.

We construct a reduction B: 1. B receives pk from the ML-DSA challenger. 2. B simulates $O_{\{\text{FIS}\}}$ for A: on query m , B calls the ML-DSA signing oracle $O_{\{\text{ML-DSA}\}}(m)$ to get σ , and returns σ (a valid PRISM-DSA signature since it passes ML-DSA verification). 3. When A outputs (m, σ) , B outputs (m, σ) .

The forgery (m, σ) passes PRISM-DSA verification, which is identical to ML-DSA verification. Therefore (m, σ) is also a valid ML-DSA forgery. This breaks ML-DSA EUF-CMA, contradicting its security under Module-SIS and Module-LWE.

The simulation is perfect: $O_{\{\text{FIS}\}}$ and $O_{\{\text{ML-DSA}\}}$ produce identically distributed signatures by Theorem 4.1, so A's view is identical. The advantage of A against PRISM-DSA equals the advantage of B against ML-DSA. $\square$

Remark 4.2 (FIS_SLOTS ≥ 1 requirement): The condition $\text{FIS_SLOTS} \geq 1$ ensures B can always answer signing queries. With $\text{FIS_SLOTS} = 64$, the simulation aborts with probability at most 2^{-23} per query (for PRISM-128, $p=0.22$); over Q queries, the total abort probability is at most $Q \cdot 2^{-23}$, which is negligible for $Q = \text{poly}(\lambda)$.

4.3 Timing Uniformity

Theorem 4.3 (Timing Uniformity): The signing time of PRISM-DSA is a function of only the parameter set and FIS_SLOTS . It does not depend on the secret key sk , the message m , or the per-signature randomness rnd .

Proof sketch: By Claims 3.1–3.3 in Section 3.2: - The loop executes exactly $\text{FIS_SLOTS} = 64$ iterations. - Each iteration executes the same sequence of arithmetic operations (NTT, invNTT, matrix-vector product, norm check, CT_SELECT), all constant-count on the coefficients. - No branch is taken on any secret-derived value.

Therefore the instruction count is:

$\text{FIS_SLOTS} \times [\text{NTT computations} + \text{matrix products} + \text{norm checks} + \text{CT_SELECT}]$

all of which have fixed instruction counts for fixed parameter sets.

□

Caveat 4.1 (Residual Channels): The proof above establishes that the control flow graph is secret-independent. Two residual channels remain:

1. **NTT data-dependent timing:** The `montgomery_reduce` and `reduce32` functions contain arithmetic that may execute in variable time on microarchitectures with variable-latency multiplication or out-of-order pipelines [Bernstein05]. Empirically, these are constant-time on x86-64 (where `IMUL` and `SHR` have fixed latency), but formal verification via tools like `ct-verif` [Almeida+16] or `haybale-pitchfork` [haybale] has not been performed.
2. **SHAKE-256 scheduling:** The SHAKE-256 implementation (via the `sha3` crate) may exhibit cache-timing effects on the Keccak sponge state, which depends on the XOF input. Since inputs to SHAKE-256 include secret-derived values (μ in particular), cache timing on SHAKE-256 is a potential channel. This is a general limitation shared by all ML-DSA-based schemes.

4.4 Remark on Theorem 4.2 (EUF-CMA)

Theorem 4.2 reduces PRISM-DSA EUF-CMA to ML-DSA EUF-CMA via a trivial simulation: the reduction answers FIS signing queries using the ML-DSA signing oracle. The argument is sound precisely because PRISM-DSA verification *is* ML-DSA verification — any valid PRISM-DSA forgery is a valid ML-DSA forgery by construction. The substantive technical content lies in **Theorem 4.1** (output distribution): the simulation is perfect only if $D_{\{FIS\}} = D_{\{ML-DSA\}}$, which Theorem 4.1 establishes (in the ROM, as a proof sketch).

A formal EUF-CMA reduction with a concrete advantage bound requires: (a) a formal proof of Theorem 4.1 with computational indistinguishability bounds under the PRF assumption on SHAKE-256, and (b) the tightness analysis of [KLS18] applied to PRISM-DSA’s signing oracle. This is planned as future work (Section 8.2 / EasyCrypt formalization).

4.5 COBALT-PQC Channels: Hint Weight and c-to Oracle

4.5.1 Hint polynomial weight channel. FIS does not affect the distribution of the public hint polynomial weight $wt(h)$. Empirical measurements using 50 independent PRISM-DSA-128 keys with 2,000 signatures each confirm:

Metric	PRISM-DSA-128	ML-DSA-44
ANOVA F	8.94	25.07
p-value	$\approx 10^{-63}$	$\approx 10^{-223}$
Spread (max – min mean)	4.96	4.56
Global mean wt(h)	62.57	62.88

Both are overwhelmingly significant ($p < 10^{-60}$) [COBALT26]. The spread is comparable, confirming that FIS selection (first-valid-slot) does not alter the marginal distribution of accepted hint weights.

Why FIS cannot fix this. FIS selects the first valid slot from 64 independent attempts. Each valid slot was accepted by the same MakeHint function with the same rejection criteria. The marginal distribution of $\text{wt}(h)$ over accepted signatures is determined by the acceptance condition and the key-specific t_0 geometry — not by how many attempts preceded acceptance.

Scope. This channel is a structural property of ML-DSA’s MakeHint function. It is not addressed by FIS, masking, or any implementation-level countermeasure. Eliminating it requires a spec-level change to the hint encoding (Section 9 of [COBALT26]).

4.5.2 $c \cdot t_0$ oracle and oracle composition (COBALT-PQC L2c). [COBALT26] characterizes which NTT-intermediate oracle access is sufficient to close the ML-DSA-44 key. A software oracle on $c \cdot t_0$ recovers t_0 (1,024 coefficients) via negacyclic matrix inversion in 16 s (L2b). Extending to a second software oracle on $c \cdot s_1$ closes the key: s_1 is recovered identically, and s_2 follows algebraically from $s_2 = 2^D \cdot t_1 + t_0 - A \cdot s_1$. All 3,072 coefficients are correctly computed in 12 s under the two-oracle software model [COBALT26, §L2c]. This does not break Module-LWE: without both oracles, recovering (s_1, s_2) from public $t = A \cdot s_1 + s_2$ remains Module-LWE-hard at 128-bit security. Hardware-traced full key recovery on Dilithium is demonstrated by Ulitzsch et al. [Ulitzsch+24] using physical side-channel measurements.

4.5.3 UseHint interval oracle: toy PoC and Phase 2

extrapolation. The prior framing of the $c \cdot t_0$ oracle (§4.5.2) assumed explicit access to the intermediate value $c \cdot t_0$. A direct instantiation — requiring no intermediate value access — arises from the UseHint verification step itself. Every accepted signature (c, z, h, t_1) is public and yields a computable constraint on t_0 :

$$V[i] = (Az - c \cdot t_1 \cdot 2^D)[i] \quad (\text{all public})$$

$$\text{UseHint}(h[i], V[i]) = \text{HighBits}(w - cs_2)[i] \rightarrow \text{center}[i] \approx ct_0[i] \pm (\gamma_2 - \beta)$$

Each coefficient of $ct_0 = M_c \cdot t_0$ (negacyclic matrix product) is thus bounded by a computable interval of width $2(\gamma_2 - \beta)$. Stacking m signatures yields $m \cdot N$ linear interval constraints over the N unknown coefficients of t_0 .

Toy proof-of-concept ($n = 4$, $q = 241$, $d = 3$, $\tau = 4$). The attack is implemented in `attacks/t0_usehint_recovery.py`. With $t_0 \in \{-3, \dots, 4\}^4$ there are $8^4 = 4,096$ candidates. Batched exhaustive elimination is applied:

Signatures used	Surviving candidates
1	$\approx 1,820$
5	≈ 347
26	≈ 3
50	1 (exact t_0)

Exact t_0 recovery is confirmed across 100 independent runs. The attack runs in < 1 s on a single CPU core. The oracle used is purely public-signature data; no side-channel measurement is required.

What the toy proves. The UseHint function is an informative interval oracle on $c \cdot t_0$. The binary check-3 framing (§4.5.2, timing signal) is a special case; the interval oracle is strictly stronger. The toy formally demonstrates that exhaustive candidate elimination converges to unique t_0 recovery from $O(50)$ public signatures.

Phase 2 extrapolation (ML-DSA-44, $n = 256$, $d = 13$). For production parameters the exhaustive approach is infeasible: the candidate space is $8192^{256} = 2^{3328}$. The $m \cdot N$ constraints define a feasibility polytope

$$P = \{ t_0 \in \mathbb{Z}^N : \|M_{\{c_i\}} \cdot t_0 - \text{center}_i\|_\infty \leq \gamma_2 - \beta, \quad i = 1..m \}$$

Recovering the unique integer point $t_0 \in P$ is an instance of the Closest Vector Problem (CVP) / Bounded Distance Decoding (BDD) on a 256-dimensional lattice. The BDD parameter is $\delta = (\gamma_2 - \beta) / \lambda_1 \approx 0.60$ for ML-DSA-44, placing the problem near the empirical hardness boundary for LLL/BDD algorithms.

Note: A pointwise NTT decomposition does NOT separate the 256 unknowns: the interval constraints are in the coefficient domain, and each coefficient of $c \cdot t_0$ is a linear combination of ALL coefficients of t_0 . No reduction to independent 1-D scalar problems is available from the UseHint oracle alone.

Parameter	Toy ($n=4$)	ML-DSA-44 ($n=256$)
Candidates	4,096	2^{3328}

Parameter	Toy (n=4)	ML-DSA-44 (n=256)
Method	Exhaustive	CVP/BDD (open)
Oracle	UseHint ✓	UseHint ✓
δ (BDD param)	—	≈ 0.60
Status	Exact recovery (50 sigs)	Open problem

Phase 2 would require a BDD solver (BKZ- β with β large enough to beat $\delta \approx 0.60$) operating on the constraint lattice. Whether this is feasible with current algorithms is an open research question.

Timing instantiation and PRISM-DSA’s partial mitigation. In standard ML-DSA (FIPS 204), the rejection check $\|ct_0\|_\infty \geq \gamma_2$ (check-3, FIPS 204 §5.5) causes a loop abort when it fires — creating a timing-observable binary signal: did ct_0 exceed the bound on this attempt? [COBALT26] identifies this as the natural timing instantiation of the $c \cdot t_0$ oracle.

PRISM-DSA eliminates this specific timing signal. `CT_NormCheck( $ct_0$ ,  $\gamma_2$ )` in the FIS Attempt function (§3.1) evaluates unconditionally across all $N=256$ coefficients without branching on the result. The check-3 outcome is folded into `slot_valid` via constant-time AND; no timing difference is observable between a slot where check-3 would have fired and one where it would not.

What PRISM-DSA does not fix. Eliminating the check-3 timing signal removes the known timing instantiation of the $c \cdot t_0$ oracle. It does not eliminate the underlying computation: ct_0 is computed unconditionally in every FIS slot, and its coefficients remain present in registers and cache lines during that computation. Power analysis, electromagnetic emanations, or cache-timing attacks that profile the ct_0 computation itself — rather than the branch outcome — constitute residual channels not addressed by FIS. The UseHint interval oracle (§4.5.3) operates on public signature data only and is not mitigated by FIS; however, its practical impact on ML-DSA-44 depends on resolving the open Phase 2 question (BDD at $\delta \approx 0.60$). Additionally, the $c \cdot s_1$ oracle (also required for L2c recovery) has no known practical timing instantiation in standard or PRISM-DSA implementations; physical instantiation via profiled template attack remains an open engineering problem [COBALT26].

4.6 Failure Probability Analysis

FIS fails when all FIS_SLOTS attempts are rejected. Since attempts are independent:

$$\Pr[\text{FIS fails}] = (1 - p_{\text{accept}})^{\{\text{FIS_SLOTS}\}}$$

For each parameter set, we decompose p_{accept} into rejection causes. p_{accept} values are derived from BDLOP18 Table 3 expected iteration counts (PRISM-128: $E[\text{iter}] \approx 4.55$ empirical / 4.25 theoretical $\rightarrow p \approx 0.22$ – 0.235 ; PRISM-192: $E[\text{iter}] \approx 5.1 \rightarrow p \approx 0.20$; PRISM-256: $E[\text{iter}] \approx 3.85 \rightarrow p \approx 0.26$; note D5 accepts faster than D2 — the parameters are not monotone in security level):

Check	PRISM-128 reject prob	PRISM-192 reject prob	PRISM-256 reject prob
$\ z\ _{\infty} \geq \gamma_1 - \beta$	~60%	~56%	~55%
$\ w_0 - \text{CS}_2\ _{\infty} \geq \gamma_2 - \beta$	~10% of passing	~12% of passing	~10% of passing
$\ ct_0\ _{\infty} \geq \gamma_2$	< 0.01% (Gaussian tail; see [COBALT26])	0% (structurally impossible: $\tau \cdot 2^{\{d-1\}} < \gamma_2$)	0% (structurally impossible: $\tau \cdot 2^{\{d-1\}} < \gamma_2$)
$\ h\ _1 > \omega$	~2% of passing	~2% of passing	~2% of passing
p_accept	~0.22	~0.20	~0.26

The dominant rejection cause is the z-norm check. The check-3 condition ($\|ct_0\|_{\infty} \geq \gamma_2$) is structurally dead at PRISM-192/256 because $\tau \cdot 2^{\{d-1\}} < \gamma_2$ by construction [COBALT26]; for PRISM-128 it can theoretically fire but $\sigma(ct_0) \approx 14,000 \ll \gamma_2 = 95,232$ makes the Gaussian tail probability negligible.

Resulting failure probabilities (computed as $(1-p)^{\{\text{FIS_SLOTS}\}}$):

FIS_SLOTS	PRISM-128 (p=0.22)	PRISM-192 (p=0.20)	PRISM-256 (p=0.26)
32	$\approx 2^{\{-11.5\}}$	$\approx 2^{\{-10\}}$	$\approx 2^{\{-14\}}$
64	$\approx 2^{\{-23\}}$	$\approx 2^{\{-21\}}$	$\approx 2^{\{-28\}}$
100	$\approx 2^{\{-36\}}$	$\approx 2^{\{-32\}}$	$\approx 2^{\{-43\}}$
200	$\approx 2^{\{-72\}}$	$\approx 2^{\{-64\}}$	$\approx 2^{\{-87\}}$

At FIS_SLOTS = 64, PRISM-128 failure probability is $2^{\{-23\}} \approx 1.2 \times 10^{\{-7\}}$ (approximately 1 failure per 8 million signing operations). For a system performing 10^6 signatures/day, this is roughly one failure per 8 days — acceptable if failures are handled with retry logic (§5.5). For applications requiring failure probabilities below $10^{\{-12\}}$, use FIS_SLOTS = 128 ($2^{\{-46\}}$).

5. Implementation

5.1 Overview

PRISM-DSA is implemented in Rust (stable, edition 2021). The implementation targets correctness and constant-time behavior. Performance optimization (SIMD NTT, AVX2 butterfly networks) is left for future work.

Dependencies: - sha3 v0.10: SHAKE-256 XOF - subtle v2.5: Constant-time conditional operations (CMOV abstraction) - rand_core v0.6: Cryptographic randomness interface - zeroize v1.7 (planned): Secret key zeroization on drop

Lines of code: ~1200 lines across 9 modules.

5.2 Module Structure

Module	Purpose	CT status
params.rs	Compile-time constants per variant	N/A
reduce.rs	Montgomery, reduce32, caddq	Yes (arithmetic only)
ntt.rs	Forward/inverse NTT, matrix product	Partial (see §5.4)
poly.rs	Polynomial operations, check_norm	Yes (for check_norm)
sample.rs	ExpandA, ExpandS, ExpandMask, SampleInBall	Partial
packing.rs	Bit-packing for all components	N/A (public data)
keygen.rs	KeyGen for all three variants	N/A
sign.rs	FIS-Sign for all three variants	Yes (FIS core)
verify.rs	Verify for all three variants	Yes (final comparison)

5.3 FIS Implementation Details

The signing function (sign128, sign192, sign256) follows the structure:

```
let mut result_bytes = [0u8; SIG_BYTES];
let mut found: u8 = 0;

for slot in 0u16..FIS_SLOTS as u16 {
    let nonce = slot * (L as u16);
    // ... compute all of: y, w, w0, w1, c, cs1, z, cs2, w0_mod,
```



```

ct0, hint
    let slot_valid = norm_check_ct(&z, GAMMA1 - BETA)
                        & norm_check_ct(&w0_mod, GAMMA2 - BETA)
                        & norm_check_ct(&ct0, GAMMA2)
                        & hint_weight_check;
    let candidate = serialize_sig(&ctilde, &z, &hint, GAMMA1,
OMEGA);
    let use_this = Choice::from(slot_valid & (found ^ 1));
    for (r, cand) in
result_bytes.iter_mut().zip(candidate.iter()) {
        *r = u8::conditional_select(r, cand, use_this);
    }
    found |= slot_valid;
}

```

Key implementation decisions: - `norm_check_ct` is a function that always iterates all coefficients - `slot_valid` is a u8 (0 or 1), accumulated with `&=` (no OR-short-circuit) - `use_this = slot_valid & (found ^ 1)` selects slot iff valid and no prior valid slot - `u8::conditional_select` is the subtle CMOV abstraction - `found |= slot_valid` latches the found state

The hint weight check uses wrapping subtraction to avoid a branch:

```

slot_valid &= (OMEGA.wrapping_sub(h_weight) >> (usize::BITS as
usize - 1)) as u8 ^ 1;

```

This evaluates to 1 iff $OMEGA \geq h_weight$ (no underflow in wrapping sense, meaning sign bit is 0), XOR'd with 1 to convert to “valid if OK”.

5.4 Constant-Time Audit

Confirmed CT (by inspection and testing):

Component	CT mechanism	Evidence
check_norm	No branch on coefficient values	Loop iterates all N coefficients
norm_check_ct	No branch on validity	<code>&=</code> accumulation
conditional_select	CMOV via subtle	subtle crate guarantee
found update	Bitwise OR	No branch
slot_valid accumulation	Bitwise AND	No branch
Final comparison (verify)	<code>subtle::ConstantTimeEq</code>	subtle crate guarantee
caddq, reduce32	Arithmetic, bit shift	No branch in source

Component	CT mechanism	Evidence
montgomery_reduce	Multiply and shift	No branch in source

NOT confirmed CT (residual channels):

Component	Concern	Impact
NTT butterfly	montgomery_reduce may have μ -arch timing on non-x86	Low (μ is public hash)
ExpandA (reject loop)	Variable-time rej_uniform	None: ρ is public
SampleInBall	Variable-time position sampling	None: $\tilde{c}$ is public
ExpandS (secret sampling)	Called at KeyGen only, not signing	Low
SHAKE-256 (sha3 crate)	Cache timing on secret-derived inputs	Shared with ML-DSA

The most significant residual channel is SHAKE-256 state-access timing on secret-derived inputs (μ is derived from tr , which hashes the public key, and the message). This is a shared limitation with all ML-DSA implementations using unverified SHAKE implementations.

5.5 Benchmark Results

Environment. Benchmarks were run on an aarch64 (ARM64) Linux 6.8.0 virtual machine, 2 cores, single-threaded, no SIMD NTT, no turbo boost. Both PRISM-DSA and the ML-DSA reference are the same pure-Rust implementation stack: this codebase vs RustCrypto’s `ml-dsa 0.1.0` crate (`cargo build --release, opt-level=3, lto=fat, codegen-units=1`). No C code, no hand-optimized assembly, no AVX2 in either implementation. This is a **Rust-vs-Rust, same-platform** comparison; the signing overhead reported below is **purely algorithmic**, not an implementation-language artifact.

PRISM-192 and PRISM-256 benchmarks are not yet available (feature-gated variants; pending implementation). The table below covers PRISM-128 only.

Operation	PRISM-128	ML-DSA-44 (Rust)	ML-DSA-65 (Rust)	ML-DSA-87 (Rust)
KeyGen	69.4 μ s	67.6 μ s	109.7 μ s	172.9 μ s

Operation	PRISM-128	ML- DSA-44 (Rust)	ML- DSA-65 (Rust)	ML- DSA-87 (Rust)
Sign	1,631 μs	163.6 μ s	58.9 μ s†	558.3 μ s†
Verify	49.4 μ s	16.2 μ s	35.1 μ s	33.5 μ s

†ML-DSA-65 and ML-DSA-87 sign times exhibit anomalous variance on this VM (see note below).

Performance analysis.

Keygen: PRISM-128 keygen (69.4 μ s) is within measurement noise of ML-DSA-44 (67.6 μ s) and faster than ML-DSA-65 (109.7 μ s). The keygen path is identical in structure to ML-DSA; no FIS overhead is present in keygen.

Signing — apples-to-apples CT comparison: The correct reference for PRISM-128 (K=4, L=4) is ML-DSA-44, which shares the same module dimensions. Comparing PRISM-128 (1,631 μ s, CT) directly against non-CT ML-DSA-44 (163.6 μ s) yields a **10 $\times$ overhead**, but this comparison is not fair: ML-DSA-44 leaks the iteration count via timing (early-exit on first valid sample), while PRISM-128 does not.

The correct reference is a hypothetical constant-time ML-DSA-44 that always completes all iterations. ML-DSA-44 averages 4.55 iterations; each complete iteration therefore costs 163.6 μ s / 4.55 $\approx$ **36.0 μ s**. Running 64 such complete iterations yields a CT-equivalent cost of **2,304 μ s**. PRISM-128 at 1,631 μ s is therefore **1.4 $\times$ faster** than a constant-time ML-DSA-44 providing the same iteration-count privacy guarantee. Stated differently: the FIS construction achieves the same CT property at 1/1.4 of the cost of the naïve “just run all N iterations” approach, because PRISM-DSA’s per-iteration cost (1,631/64 $\approx$ 25.5 μ s) is lower than a complete ML-DSA-44 iteration (36.0 μ s). The theoretical FIS factor is 64 / (1/p_{accept}) = 64 / 4.55 $\approx$ 14.1 $\times$; the measured 10 $\times$ is lower because PRISM-DSA’s CT selection loop overhead (conditional_select over 64 candidates) is dominated by the per-slot compute.

Verify: PRISM-128 verify (49.4 μ s) is **3.0 $\times$ slower** than ML-DSA-44 (16.2 μ s). The verify path is structurally identical to ML-DSA (same matrix-vector operations, same hint check, same challenge reconstruction); no FIS overhead is present. The gap is a pure **implementation delta**: our verify.rs performs the same operations as ml-dsa 0.1.0 but without its micro-optimizations. This gap is expected to close with equivalent NTT optimization; it does not represent an algorithmic cost of PRISM-DSA.

VM measurement note — ML-DSA-65 and ML-DSA-87: The ML-DSA-65 sign time (58.9 μ s) and ML-DSA-87 sign time (558.3 μ s) exhibit anomalous variance on this 2-core VM under shared load. ML-DSA-65 appearing faster than ML-DSA-44 despite larger module dimensions ($K=6, L=5$ vs $K=4, L=4$) is inconsistent with the algorithmic cost model and is attributed to VM scheduler interference with the stochastic rejection-sampling loop. PRISM-128 numbers are stable across runs because FIS always executes exactly 64 iterations regardless of VM load. All ML-DSA cross-variant comparisons in this paper should be treated as indicative; the PRISM-128 vs CT-equivalent ML-DSA-44 comparison (same dimensions, same platform) remains valid.

SIMD projection: Neither implementation uses SIMD NTT. If PRISM-DSA's NTT were optimized to match an AVX2/NEON implementation ($\sim 5\text{--}8\times$ NTT speedup [pq-crystals/dilithium]), projected PRISM-128 signing would reach $\approx 200\text{--}300$ μ s. SIMD applies equally to ML-DSA, so the algorithmic FIS ratio is preserved; only absolute latency decreases.

FIS_SLOTS = 64 balances failure probability and performance: the failure probability is 2^{-23} (≈ 1 in 8×10^6). Reductions to 32 give $2^{-11.5}$ (unacceptable for high-volume deployments); 100 gives 2^{-36} at 56% more computation. On failure, the caller must retry with fresh randomness — at the application level this re-introduces variable signing time; deployments requiring strict wall-clock CT bounds should use $\text{FIS_SLOTS} \geq 128$.

6. Parameter Selection

6.1 Choosing FIS_SLOTS

The FIS_SLOTS parameter controls three properties:

1. **Failure probability:** $P(\text{failure}) = (1 - p_{\text{accept}})^{\{\text{FIS_SLOTS}\}}$
2. **Signing time:** $T_{\text{sign}} = \text{FIS_SLOTS} \times T_{\text{one_attempt}}$
3. **Security margin:** More slots provide more timing uniformity headroom

The choice $\text{FIS_SLOTS} = 64$ is motivated by:

Failure probability: $2^{-23} \approx 1.2 \times 10^{-7}$ (approximately 1 failure per 8 million signing operations). For a system performing 10^6 signatures per day, expected failure rate is once every ~ 8 days. This is acceptable if the signing interface handles FAILURE by retrying with fresh randomness; unacceptable if failures are surfaced to users or if the retry itself must be timing-uniform. For higher assurance, $\text{FIS_SLOTS} = 100$ gives 2^{-36} (≈ 1 failure per 70 billion operations) and $\text{FIS_SLOTS} = 200$ gives 2^{-72} .

Performance: On a pure-Rust Rust-vs-Rust baseline, PRISM-128 signs in 1.46 ms vs non-CT ML-DSA-65 at 175 μ s ($8.3\times$ overhead); the apples-to-apples CT comparison gives $5.5\times$ faster than a hypothetical constant-time ML-DSA-65 (see §5.5). The 1.46 ms signing time is acceptable for interactive authentication (latency budget typically 100–500 ms). For high-throughput applications (>1000 TPS requires <1 ms), use the projected SIMD build ($\sim 200\text{--}300$ μ s) or accept the unoptimized tradeoff.

Power-of-two: $\text{FIS_SLOTS} = 64 = 2^6$ is a minor convenience for $L = 4$ (PRISM-128, where $\text{nonce} = \text{slot} \times 4$ is a single shift). For PRISM-192 ($L=5$) and PRISM-256 ($L=7$), L is not a power-of-two; the nonce arithmetic uses a multiply in all cases.

6.2 Comparison with ML-DSA Parameter Sets

The underlying parameter sets ($K, L, \eta, \tau, \beta, \gamma_1, \gamma_2, \omega$) are inherited unchanged from ML-DSA-44/65/87. The rationale for these parameters is documented in [FIPS204] and [BDLOP18]. PRISM-DSA does not modify these parameters, so the security level claims and hardness assumptions are identical to ML-DSA.

The choice of γ_1 and γ_2 affect p_{accept} : larger γ_1 (relative to β) increases acceptance probability. For PRISM-128, $\gamma_1 = 2^{17}$ and $\beta = 78$ gives $(\gamma_1 - \beta)/\gamma_1 = 1 - 78/131072 \approx 0.9994$, meaning $\sim 99.94\%$ of individual coefficients pass the z-norm check. The joint probability over all $LN = 1024$ coefficients gives the $\sim 60\%$ z-rejection rate observed empirically.

6.3 Alternative: FIS with Bounded Slots as Distinct Algorithm Family

An interesting extension is to define $\text{FIS}_{\{\text{SLOTS}, T\}}$ as a family parameterized by both the number of slots and a time limit T (in seconds). If signing exceeds T , return FAILURE. This provides a wall-clock upper bound on signing time independent of implementation performance, which may be useful for real-time systems. We leave this extension for future work.

7. Comparison with Related Work

7.1 ML-DSA / CRYSTALS-Dilithium

ML-DSA [FIPS204, BDLOP18] is the direct ancestor of PRISM-DSA. All parameter sets, ring structure, and verification algorithms are inherited from ML-DSA. The sole difference is the signing algorithm (unbounded loop vs FIS). See Section 9 of the specification for a detailed comparison table.

The closest prior approach to timing-safe ML-DSA signing is masking [Azouaoui+23], which provides constant-time behavior against power and cache side-channels at $\sim 3\text{-}5\times$ software overhead for 2nd-order masking. FIS targets specifically the rejection-count timing channel and achieves it at $8.3\times$ vs non-CT ML-DSA-65, or equivalently $5.5\times$ faster than a constant-time ML-DSA-65 (see §5.5 for the distinction). FIS and masking are orthogonal: a masked PRISM-DSA would address both channels simultaneously.

7.2 Falcon

Falcon [FALCON] is an NIST-selected signature scheme based on NTRU lattices and Fast Fourier Sampling. Falcon achieves smaller signatures than ML-DSA (666 bytes vs 2420 bytes for Level 1 security) but its signing algorithm involves floating-point Gaussian sampling, which is notoriously difficult to implement in constant time. Multiple timing issues have been documented for Falcon's floating-point Gaussian sampler [Prest15, FALCON]; the Falcon specification §3.12 addresses constant-time implementation requirements in detail [FALCON]. PRISM-DSA does not compete with Falcon on signature size but avoids the Gaussian sampler complexity entirely.

7.3 HAWK

HAWK [HAWK] is a lattice signature scheme using module NTRU lattices with compact signatures and a constant-time signing algorithm based on Gaussian sampling with rejection. HAWK's constant-time property comes from a different design choice (fixed Gaussian parameters) rather than fixed iteration count. A full comparison of timing uniformity properties is left for future work.

7.4 SPHINCS+

SPHINCS+ [SPHINCS+] is a hash-based signature scheme (stateless). It achieves unconditional security without lattice assumptions and has a fully deterministic signing algorithm. SPHINCS+ signing is inherently constant-time (deterministic tree

traversal) but produces large signatures ($\sim 8\text{--}50$ KB) and is significantly slower. PRISM-DSA is not an alternative to SPHINCS+ but occupies a different point in the security-size-speed tradeoff space.

7.5 Dilithium-G

Dilithium-G [Karabulut+21] is a proposed variant of Dilithium using a Gaussian signing distribution (instead of uniform) to reduce rejection rates. This improves performance but does not address the variable-time issue (Gaussian sampling is itself variable-time). FIS is orthogonal: it could in principle be applied to Dilithium-G to achieve both lower expected iteration count and fixed-time execution.

7.6 Masking Countermeasures

An alternative approach to timing side-channels in FSwa is masking: splitting secret values into multiple shares and operating on shares [ISW03]. Masking eliminates both timing and power side-channels simultaneously and is well-studied for ML-DSA [Azouaoui+23, Cassiers+23]. Modern 2nd-order masked ML-DSA implementations achieve $\sim 3\text{--}5\times$ software overhead [Azouaoui+23] — not the $10\text{--}100\times$ of earlier proposals. FIS targets the rejection-count timing channel only, at $8.3\times$ vs non-CT ML-DSA-65 ($5.5\times$ faster than CT-equivalent ML-DSA-65 — see §5.5 for the correct comparison). FIS and masking are complementary: a masked PRISM-DSA would cover both channels.

7.7 Masking-Friendly and Fixed-Time Lattice Signatures

Several recent schemes in the NIST additional standardization process explore related design goals. Raccoon (NIST alternate candidate, ~ 2024) is designed specifically for masking-friendly signing, using a different approach from FIS — it modifies the acceptance distribution rather than iterating to a fixed slot count. HAETAE (ASIACRYPT 2023) explores tighter rejection sampling bounds, reducing expected iteration count. A detailed comparison of timing properties against these schemes is left for future work; we note that FIS is orthogonal to both (it could in principle be applied on top of either to bound iteration count).

Citation note: Raccoon and HAETAE citations are pending verification of final publication venues; see NIST PQC additional signatures process documentation.

8. Conclusion and Future Work

8.1 Summary

We presented PRISM-DSA, a post-quantum signature scheme that achieves constant-time signing by replacing the unbounded rejection loop of ML-DSA with a Fixed-Iteration Signing construction. The scheme is:

- **Secure:** EUF-CMA under Module-SIS and Module-LWE (same as ML-DSA)
- **Compatible:** PRISM-DSA signatures pass ML-DSA verification
- **Timing-uniform:** Fixed loop count, CT selection, CT norm checks
- **Practical:** 8.3× overhead vs non-CT ML-DSA-65; 5.5× faster than CT-equivalent ML-DSA-65 (§5.5)
- **Implemented:** Pure Rust, no unsafe code, three parameter variants

8.2 Future Work

EasyCrypt formalization: The key claims (Theorems 4.1, 4.2, 4.3) are argued informally. Formal machine-checked proofs via EasyCrypt [Barthe+11] would significantly strengthen the security assurance. The output distribution claim (Theorem 4.1) is likely the most tractable starting point, as it reduces to a simple independence argument over the SHAKE-256 random oracle.

SIMD NTT optimization: The current NTT implementation processes coefficients sequentially. An AVX2 or AVX-512 optimized NTT (as in the ML-DSA reference implementation [pq-crystals/dilithium]) would reduce per-iteration cost by 5–8×, giving projected PRISM-128 signing times of ~200–300 μs.

FPGA implementation: Fixed-iteration signing is natural for hardware: the 64-slot FIS loop maps directly to a pipeline of 64 concurrent signing engines. On an FPGA, all 64 attempts could execute in parallel, potentially matching ML-DSA’s latency while maintaining timing uniformity. This is of interest for high-throughput HSM applications.

Formal CT verification: Full constant-time verification of the NTT and SHAKE-256 components requires formal tools. We intend to apply ct-verif [Almeida+16] or haybale-pitchfork [haybale] to the Rust implementation.

Key Zeroization: The current implementation does not implement Zeroize on drop for SecretKey structs. This allows secret key material to remain in heap memory after use. Proper zeroization via the zeroize crate is required for production use.

Higher FIS_SLOTS variants: For PRISM-128 ($p=0.22$), the corrected failure probabilities are: $\text{FIS_SLOTS}=128 \rightarrow 2^{-46}$; $\text{FIS_SLOTS}=200 \rightarrow 2^{-72}$. These are appropriate for certificate authority or HSM applications requiring failure probability below 10^{-12} .

References

- [FIPS204] National Institute of Standards and Technology. *Module-Lattice-Based Digital Signature Standard (ML-DSA)*. FIPS 204. August 2024. <https://doi.org/10.6028/NIST.FIPS.204>
- [BDLOP18] Shi Bai, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, Damien Stehlé. *CRYSTALS-Dilithium Algorithm Specifications and Supporting Documentation*. 2021 (version 3.1). <https://pq-crystals.org/dilithium/>
- [Lyub09] Vadim Lyubashevsky. *Fiat-Shamir with Aborts: Applications to Lattice and Factoring-Based Signatures*. ASIACRYPT 2009. https://doi.org/10.1007/978-3-642-10366-7_35
- [KLS18] Eike Kiltz, Vadim Lyubashevsky, Christian Schaffner. *A Concrete Treatment of Fiat-Shamir Signatures in the Quantum Random-Oracle Model*. EUROCRYPT 2018. https://doi.org/10.1007/978-3-319-78372-7_18
- [FALCON] Pierre-Alain Fouque, Jeffrey Hoffstein, Paul Kirchner, Vadim Lyubashevsky, Thomas Pornin, Thomas Prest, Thomas Ricosset, Gregor Seiler, William Whyte, Zhenfei Zhang. *Falcon: Fast-Fourier Lattice-based Compact Signatures over NTRU*. NIST submission 2020. <https://falcon-sign.info/>
- [HAWK] Léo Ducas, Eamonn W. Postlethwaite, Ludo N. Pulles, Wessel P.J. van Woerden. *HAWK: Module LIP Makes Lattice Signatures Fast, Compact and Simple*. ASIACRYPT 2022. <https://eprint.iacr.org/2022/1155>
- [SPHINCSP] Jean-Philippe Aumasson, Daniel J. Bernstein, Ward Beullens, Christoph Dobraunig, Maria Eichlseder, Scott Fluhrer, Stefan-Lukas Gazdag, Andreas Hülsing, Panos Kampanakis, Stefan Kölbl, Tanja Lange, Martin M. Lauridsen, Florian Mendel, Ruben Niederhagen, Christian Rechberger, Joost Rijneveld, Peter Schwabe. *SPHINCS+*. NIST submission 2022. <https://sphincs.org/>
- [Ravi+24] Prasanna Ravi, Dirmanto Jap, Shivam Bhasin. *Evaluation of ML-DSA (Dilithium) Side-Channel Resilience and Countermeasures*. NIST 5th PQC Standardization Conference.

2024. <https://csrc.nist.gov/csrc/media/Events/2024/fifth-pqc-standardization-conference/documents/papers/evaluation-ml-dsa-dilithium.pdf>

[Berzati+23] Alexandre Berzati, Andersson Calle Viera, Maya Chartouny, Steven Madec, Damien Vergnaud, David Vigilant. *Exploiting Small Leakages in Masks to Turn a Second-Order Attack into a First-Order Attack*. TCHES 2023(3). <https://doi.org/10.46586/tches.v2023.i3.1-24>

[Ulitzsch+24] Vincent Ulitzsch, Soundes Marzougui, Juliane Krämer, Jean-Pierre Seifert. *A Side-Channel Assisted Cryptanalytic Attack on CRYSTALS-Dilithium*. PKC 2024. https://doi.org/10.1007/978-3-031-57722-2_8

[Pei16] Chris Peikert. *A Decade of Lattice Cryptography*. Foundations and Trends in Theoretical Computer Science 10(4). 2016. <https://doi.org/10.1561/04000000074>

[Bernstein05] Daniel J. Bernstein. *Cache-timing attacks on AES*. 2005. <https://cr.yp.to/antiforgery/cachetiming-20050414.pdf>

[Almeida+16] José Bacelar Almeida, Manuel Barbosa, Gilles Barthe, François Dupressoir, Michael Emmi. *Verifying Constant-Time Implementations*. USENIX Security 2016. <https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/almeida>

[haybale] Sam Lerner. *haybale-pitchfork: Checking for constant-time code using symbolic execution*. 2020. <https://github.com/PLSysSec/haybale-pitchfork>

[Intel23] Intel Corporation. *Intel 64 and IA-32 Architectures Software Developer's Manual*. Volume 3, 2023. <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>

[ARM23] Arm Limited. *Arm Architecture Reference Manual for A-profile architecture*. DDI 0487, 2023. <https://developer.arm.com/documentation/ddi0487/latest>

[Prest15] Thomas Prest. *Gaussian Sampling in Lattice-Based Cryptography*. PhD thesis, Ecole Normale Supérieure, 2015. <https://theses.hal.science/tel-01245066>

[Pornin20] Thomas Pornin et al. *Falcon: Fast-Fourier Lattice-based Compact Signatures over NTRU — Specification v1.2*. Section 3.12: constant-time implementation requirements. <https://falcon-sign.info/falcon.pdf> [Note: same authors as [FALCON]; cited separately for the specific §3.12 constant-time discussion. Merge with [FALCON] on next revision.]

[Karabulut+21] Emre Karabulut, Erdem Alkim, Aydin Aysu. *Single-Trace Side-Channel Attacks on ω -Small Polynomial Sampling: With Applications to NTRU, NTRU Prime, and CRYSTALS-Dilithium*. HOST 2021. <https://doi.org/10.1109/HOST49136.2021.9702283>

[ISW03] Yuval Ishai, Amit Sahai, David Wagner. *Private Circuits: Securing Hardware against Probing Attacks*. CRYPTO 2003. https://doi.org/10.1007/978-3-540-45146-4_27

[Azouaoui+23] Melissa Azouaoui, Olivier Bronchain, Clément Hoffmann, Yulia Kuzovkova, Tobias Schneider, François-Xavier Standaert. *Leveling Dilithium against Leakage: Revisited Sensitivity Analysis and Improved Implementations*. TCHES 2023(4). <https://doi.org/10.46586/tches.v2023.i4.423-459>

[Cassiers+23] Gaëtan Cassiers, Barbara Gigerl, Stefan Mangard, Charles Momin, Praveen Kumar Vadnala. *Randomness Generation for Secure Hardware Masking — Unrolled Trivium to the Rescue*. TCHES 2023(4). <https://doi.org/10.46586/tches.v2023.i4.100-137>

[Barthe+11] Gilles Barthe, Benjamin Grégoire, Santiago Zanella Béguelin. *Formal Certification of Code-Based Cryptographic Proofs*. POPL 2009; extended EasyCrypt system described in *Computer-Aided Security Proofs for the Working Cryptographer*. CRYPTO 2011. https://doi.org/10.1007/978-3-642-22792-9_5

[COBALT26] Dominik Blain. *Structural Leakage of to Statistics through Hint Polynomial Weight in FIPS 204 Signatures*. COBALT-PQC Technical Report. 2026. <https://eprint.iacr.org/> (pending submission)

PRISM-DSA is a research prototype. Formal security proofs are pending. Not for production use without independent security review.